

Ablauforganisation

Note

Ablauforganisation = Aneinanderreihung systeminterner Elemente (Arbeitsabläufe) zur Zielerreichung

Zweck:

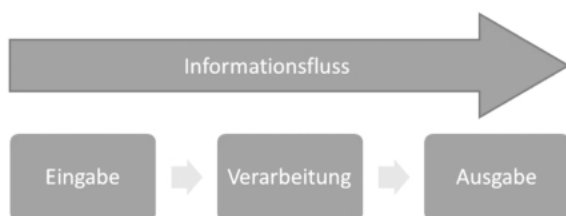
- Arbeitsvorgänge organisieren
- Verantwortlichkeiten formalisieren
 - hier eher nicht "Wer ist der Chef" sondern eher "was macht der Chef"
- Rationalisierung ermöglichen
- Standardisierung anstreben

Prozess

Definitionen

Prozessdefinitionen (Beispiele):

- kontinuierliche Entwicklung mit vielen Änderungen
[Webster's] „E
- Menge von Tätigkeiten, die Eingaben in Ausgaben transformieren
[ISO 90003]
- Sequenz von Schritten zur Zweckerfüllung
[IEEE]
- Methoden, Praktiken und Vorschriften zur Softwareentwicklung
[SEI]

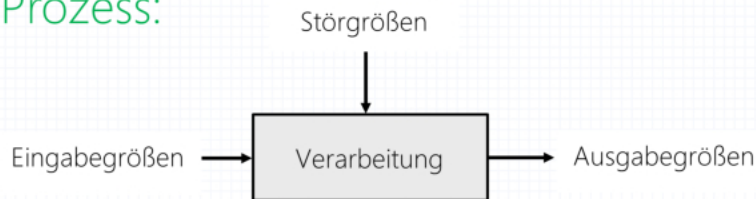


„E V A – Prinzip“

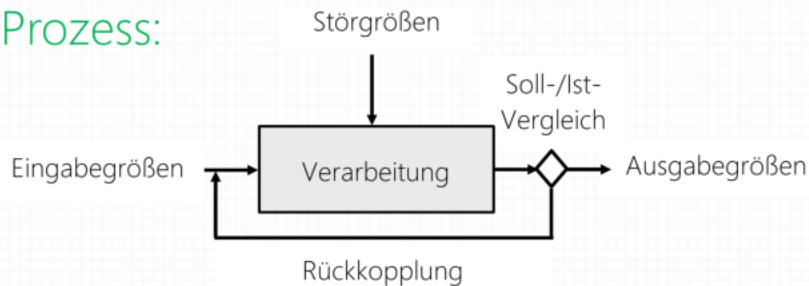
Gesteuerter Prozess vs. geregelter Prozess

Gesteuerter vs. geregelter Prozess

Gesteuerter Prozess:



Geregelter Prozess:



-> In der Softwareentwicklung benötigt man **geregelte** Prozesse!

Beispiel:

- Stromheizer auf Stufe 2 ohne Thermostat
 - heizt - egal wie heiß es ist
- mit Thermostat
 - heißt, prüft ob es warm genug ist und ggf. heißt es weiter oder auch nicht

Definierter vs. empirischer Prozess

Definierter Prozess

- Aktivitäten vorab bekannt und verstanden
- Ergebnis vorhersehbar
- Prozess wiederholbar
- Beginn und Ende vorab festlegbar

Beispiel:

- schrittweise Verfeinerung
- Alltagsbeispiel: Auto bauen - man kann einen perfekten Plan haben, den man so umsetzen kann.

Empirischer Prozess

- komplex und unvorhersehbar
- nicht vollständig verstanden
- nicht durchgehend definierbar

Beispiel: **iterativ-inkrementelles Adaptieren**

- **iterativ**: Bestehendes wird immer weiter verbessert
- **inkrementell**: Bestehendes wird um neue Funktionen erweitert
- **Adaptieren**: Anpassung
Alltagsbeispiel: - neue Art von Auto erfinden (z. B. autonom fahrend) - so komplex und unvorhersehbar dass man immer nur ein paar Schritte gehen kann.

Note

Auch empirische Prozesse können erfolgreich geregelt werden, selbst wenn sie nicht vollständig verstanden werden!

Im Autobeispiel heißt das, dass immer wieder die Funktionsweise überprüft wird, und die Verarbeitung wiederholt wird, bis das Auto tatsächlich autonom fährt.

Erfolgreiche Regelung durch:

- häufige Inspektionen
- laufende Anpassung
- Trennung der Steuerung der Eingabe von Störfaktoren

Empirischer Softwareentwicklungsprozess

Das ist die Übertragung der Theorie vom empirischen Prozess auf die Softwareentwicklung.

Ein großes Softwareprojekt ist komplex und unvorhersehbar --> empirischer Prozess. Wie arbeitet man an so einem Prozess?

- kontinuierlich planen
- frühzeitig realisieren, sobald genügend Anforderungen bekannt; „the art of the possible“
- keine Stabilität erwarten
- auf Änderungen von Anforderungen oder Rahmenbedingungen rasch reagieren

SW-Entwicklung soll ein geregelter Prozess sein.

Also:

Info

Rückkopplung durch Bewertung der Zwischenprodukte durch den Kunden ist entscheidend für den weiteren Projektverlauf (emergentes Verhalten) sowie die Ausrichtung des Produkts (Value-added Software Development).

Def.:

- emergentes Verhalten (flexibel reagieren auf die Wünsche des Kunden und den Projektverlauf davon abhängig machen)
- Value-added Software Development (Programmiert wird, was dem Kunden einen Nutzen bringt)

Entwicklung der Prozessbedeutung

Verschiebung der **Bedeutung des Prozesses** im Lauf der Zeit:

- **normative Bedeutung** (ab ~1985):
Es gibt einen „besten“ Prozess.
 - **transformierende Bedeutung** (ab ~1995):
Prozesse brauchen konkrete Umsetzungsrichtlinien.
 - **wertgenerierende Bedeutung** (ab ~2005):
Der Prozess wird durch den Wert des (Teil-)Produkts für den Kunden gesteuert.
- wertgenerierende Bedeutung
 - der agile Wendepunkt
 - der Wert für den Kunden steht im Vordergrund
 - Das Produkt steuert den Prozess

Kernaufgaben der Ablauforganisation

Aufgaben:

- Prozesse in Gang setzen
- Prozesse analysieren
- Prozesse verbessern

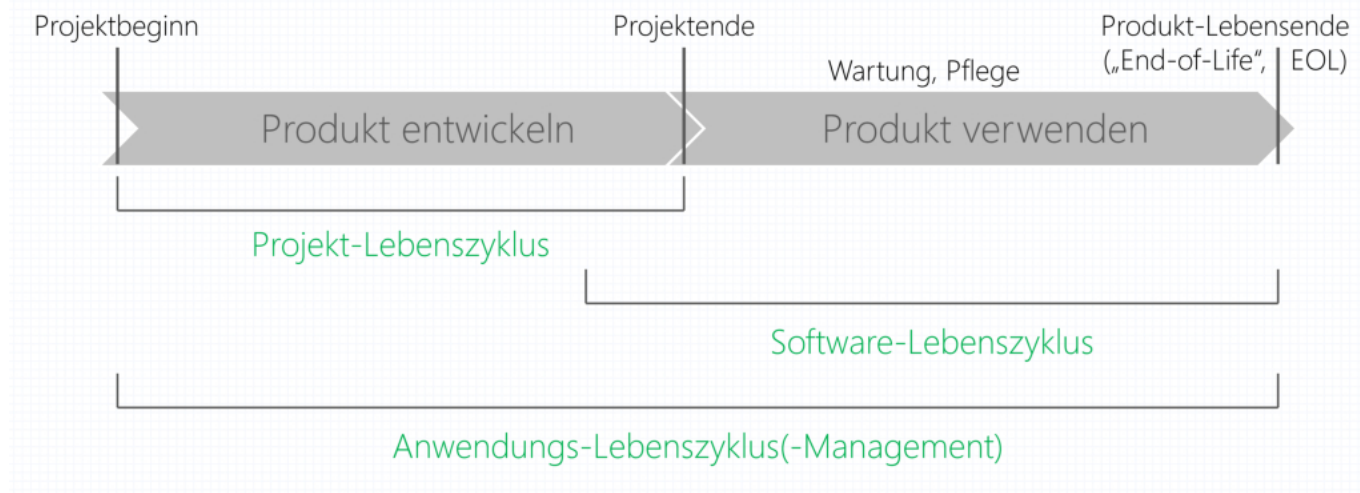
Prozesse sind produktbezogen (aus Projektsicht statisch) oder prozessbezogen (aus Projektsicht dynamisch).

Prozesse vs. Lebenszyklen

Es gibt unterschiedliche Sichtweisen darauf, wann ein Prozess beendet ist.

- **Projekt-Lebenszyklus**
 - Ordnung der zeitlichen Abfolge der Aktivitäten (von der Entwicklung bis zum Einsatz)
 - Analogie Haus: Hausbau ist mit Schlüsselübergabe abgeschlossen
- **Software-Lebenszyklus**
 - Ordnung der zeitlichen Abfolge der Aktivitäten (von der Entwicklung über den Einsatz bis zum Ende der Benutzung)
 - Analogie Haus: Haus-Prozess dauert vom Bau, über die Wartung bis zum Abriss.
- **Anwendungs-Lebenszyklus-Management** (Application Lifecycle Management; ALM)
 - Moderne Sicht – Trennung vor/nach Projektende wird unscharf (kontinuierliche Entwicklung und Freigabe während des gesamten Lebenszyklus des Produkts)
 - ständige Verbesserung gemeinsam mit dem Kunden
 - Analogie Haus: Das Haus wird ständig ausgebaut, ständiger Verbesserungsprozess

Prozesse vs. Lebenszyklen (II)



heutzutage verschwimmt der Bereich zwischen Produkt entwickeln und Produkt verwenden; es wird eher die Software immer weiter entwickelt

Altes Vorgehensmodell:

Dokumentation der Ablauforganisation -> wurde Ende der Neunziger Jahre immer umfangreicher

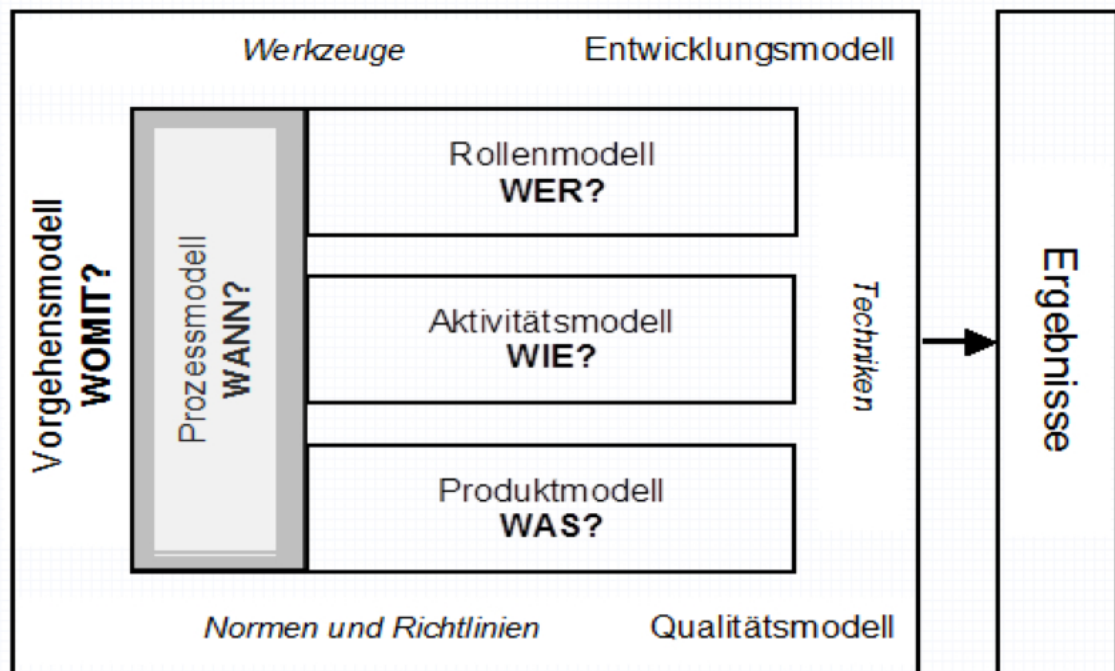
- alles wurde geplant, das kostete viel Zeit und ist wenig flexibel
-

- kleine Anforderungen können sich ändern
Gegenbewegung war die agile Softwareentwicklung - flexibel, hin zur Kommunikation.

Das **neue Vorgehensmodell** ist die systematische Gliederung einer Lösung.
d.h.

- Projektmanager versuchen zunehmend, aus Prozesserfahrungen zu lernen.
- Prozessmuster (Patterns) sind bewährte Praktiken, die induktiv aus Prozesserfahrungen erarbeitet wurden.
- Es gibt auch Sammlungen von Negativbeispielen (Anti-Patterns).
- *man plant nicht alles bis ins letzte Detail, sondern kann durch Erfahrung auf neue Situationen reagieren*

Komponenten eines modellbasierten Vorgehens



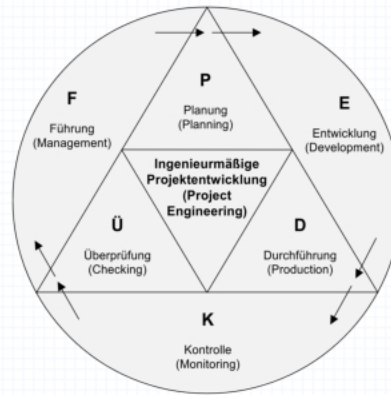
Prozessorientiertes Vorgehen

Prozessmodell unterteilt Vorgehen in überschaubare Abschnitte:

Prozessmodell unterteilt Vorgehen in überschaubare Abschnitte,

- schrittweise Planung,
- Durchführung und
- Überprüfung,

mit Rückkopplungen.



Prozessorientiertes Vorgehen = Vorgehen unter Beachtung eines expliziten Prozessmodells.

Vorgehensmethoden

Unterscheidung: Vorgehensprozesse werden durch Vorgehensmethoden umgesetzt.
Oft Erfolg nur durch Kombination verschiedener Methoden!

Sequenzielles Vorgehen

Phasenmodell



abgeleitet von der Organisation der Entwicklung allgemeiner technischer Systeme -> veraltet

Eins nach dem anderen, starr

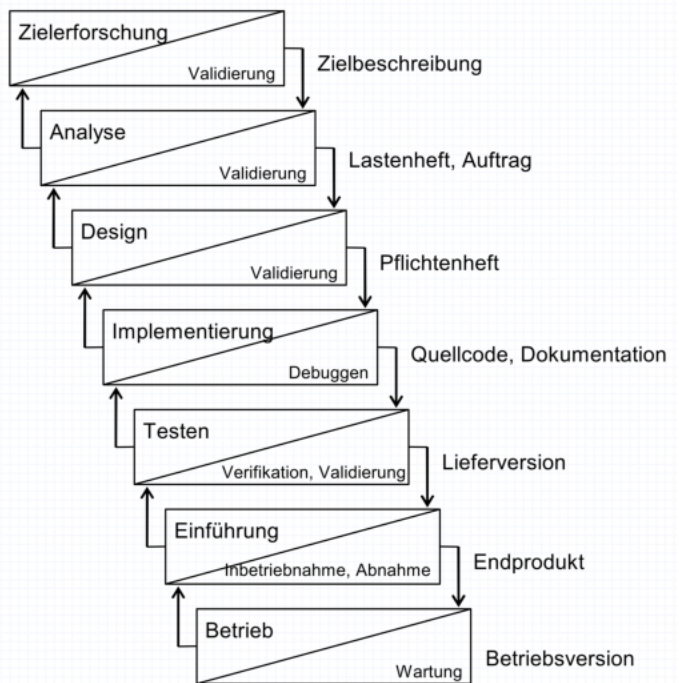
Inkrementelles Vorgehen - Wasserfallmodell

- inkrementell: funktionale Teilerweiterung, step by step
- kaum Flexibilität, Änderungswünsche teuer

Vorgehensmethoden – Inkrementelles Vorgehen (I)

Wasserfallmodell

kaum Flexibilität,
Änderungswünsche teuer



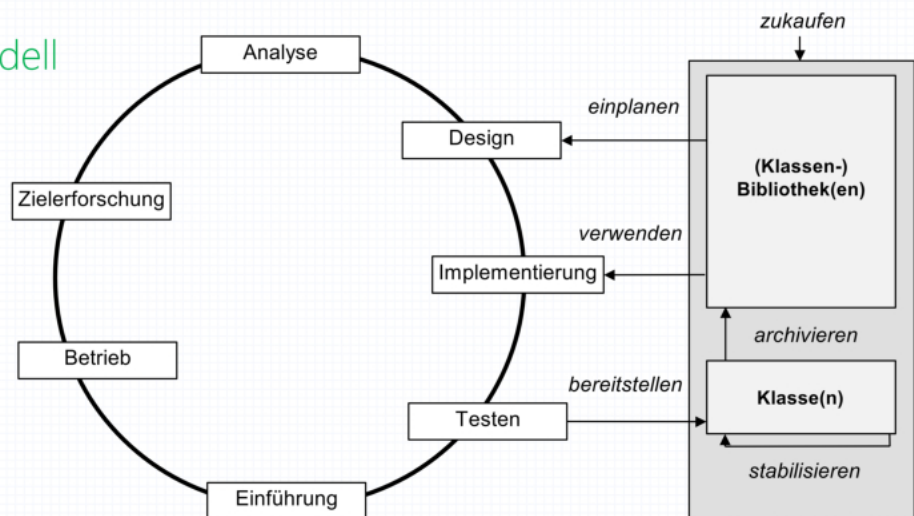
Inkrementelles Vorgehen - Objektorientiertes Modell

Es sollen, wenn möglich Libraries verwendet werden - diese werden selbst erstellt oder auch zugekauft und ermöglichen die Wiederverwendung von Code für verschiedene Projekte

Vorgehensmethoden – Inkrementelles Vorgehen (II)

Objektorientiertes Modell

„Tool-Smithing“,
Wiederverwendung setzt Wiederverwendbarkeit voraus!



Iteratives Vorgehen - kontinuierliche Verbesserung

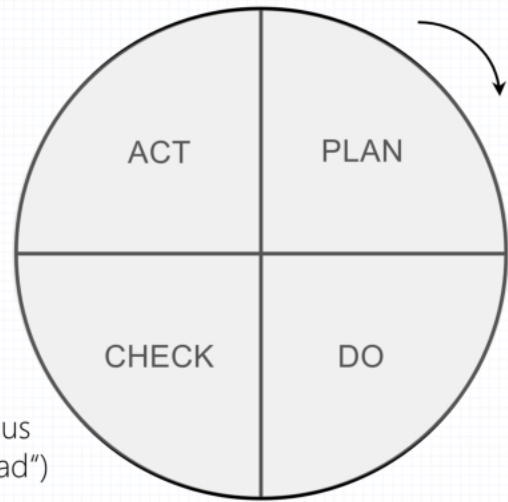
iterativ = wiederholte, schrittweise Verbesserung bereits vorhandener Funktionen

Vorgehensmethoden – Iteratives Vorgehen (I)

Grundidee: **kontinuierliche Verbesserung** (PDCA-Zyklus) [Deming]
(iterativ = wiederholt!)

Erstellung eines neuen Produkts
in jeder Iteration,
aber Aufbauen auf
erarbeitetem Wissen!

PDCA-Zyklus
(„Deming-Rad“)



Praxis: z.B. Suchfunktion

1. sie funktioniert
2. sie funktioniert auch bei Tippfehlern
3. sie funktioniert auch bei Tippfehlern schneller als je zuvor
4. usw.

Man versucht dabei also so schnell wie möglich ein Produkt zum Kunden zu bringen, um schnell herauszufinden, wo nachgebessert werden muss, bzw. ob der Kunde das so richtig gemeint hat.

Motivation: Fehlerbehebungskosten steigen im Laufe der Projektentwicklung überproportional an.

D.h. je später man einen Fehler findet, desto teurer wird die Behebung.

Vorgehensmethoden - Iteratives Vorgehen

Vorgehensmethoden – Iteratives Vorgehen (II)

Motivation: **Fehlerbehebungskosten** steigen im Laufe der Projektentwicklung überproportional an.

Beispiel: Fehlerbehebungskosten bei Siemens PSE Österreich (ca. 2005)

Bereich	Analyse	Design	Codieren	Debuggen	Testen	Betrieb
Relative Anzahl der entstandenen Fehler	10%	40%	50%	-	-	-
Relative Anzahl der erkannten Fehler	3%	5%	7%	25%	50%	10%
Kosten pro Fehlerkorrektur (€)	250	250	250	1000	3000	12500

Info

Daher macht man bei dieser Methode Prototyping. Im Speziellen: **Exploratives Prototyping**, um so schnell wie möglich Feedback zu erhalten, danach kommt eine neue Iteration.

Vorgehensmethoden – Iteratives Vorgehen (III)

Exploratives Prototyping

Entwicklung so früh wie möglich – rasches Feedback vom Kunden, danach neue Iteration

Prototypen und Prototyping:

- Prototyp = (günstiges) ausführbares Modell eines Produkts
- Prototyping = Arbeiten zu Herstellung von Prototypen

Arten des explorativen Prototyping:

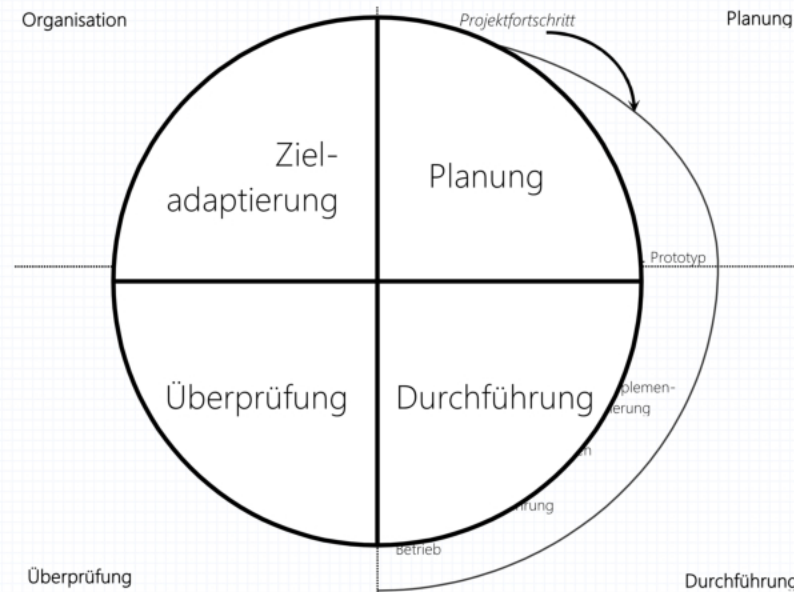
- Revelationäres Prototyping (eher Kunden-orientiert)
- Experimentelles Prototyping (eher Entwickler-orientiert)

- ## Vorgehensmethoden - Iteratives Vorgehen



Spiralmodell (III): (rein) revolutionäres Prototyping

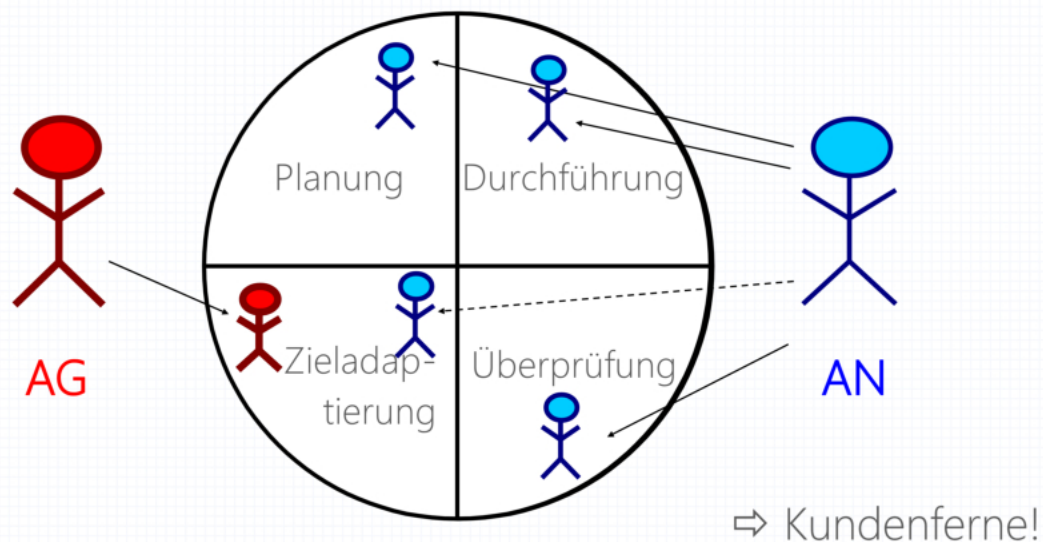
Einbringen von Prototyping-Konzepten [Boehm]



Spiralmodell (IV): (rein) revolutionäres Prototyping

Spiralmodell (IV): (rein) revolutionäres Prototyping

„klassisches“ Spiralmodell



Vorgehensmethoden – Iterativ-inkrementelles Vorgehen (I)

Grundidee: Evolutionäres Vorgehen – Entwicklung in Zyklen, jeweils auf den Ergebnissen des vorherigen Zyklus aufbauend

Es ist also die Wiederholung (iterativ) von Erweiterungen (inkrementell) - so zu sagen step-by-step development

d.h. 1. Planung 2. Durchführung 3. Überprüfung 4. Zieladaptierung ist eine Iteration. Dann kommt ein neues Inkrement.

iterativ-inkrementelles Adaptieren

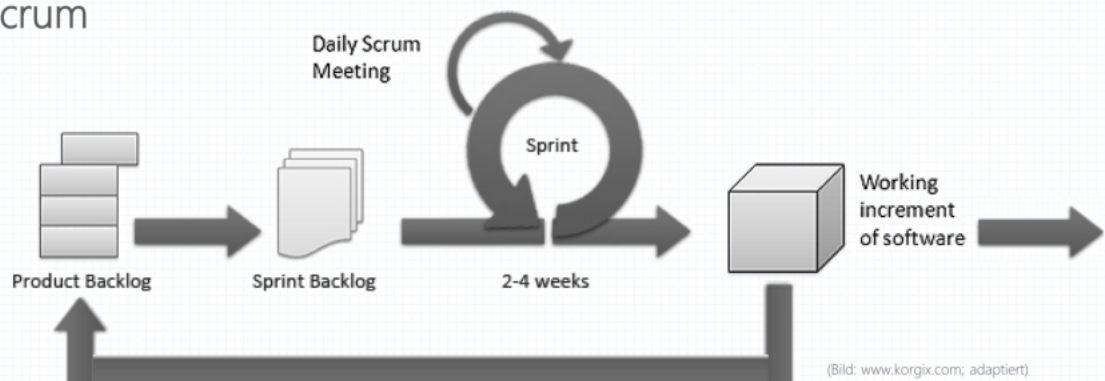
- **iterativ**: Bestehendes wird wiederholt verbessert (wiederholung)
- **inkrementell**: Bestehendes wird um neue Funktionen erweitert
- **Adaptieren**: Anpassung

Alltagsbeispiel: - neue Art von Auto erfinden (z. B. autonom fahrend) - so komplex und unvorhersehbar dass man immer nur ein paar Schritte gehen kann.

Vorgehensmethoden – Iterativ-inkrementelles Vorgehen (I)

Grundidee: **Evolutionäres Vorgehen** – Entwicklung in Zyklen, jeweils auf den Ergebnissen des vorherigen Zyklus aufbauend

Beispiel: Scrum



Iterativ-inkrementelles Vorgehen: Evolutionäres Prototyping

Vorgehensmethoden – Iterativ-inkrementelles Vorgehen (II)

Evolutionäres Prototyping

Prototypen werden inkrementell weiterentwickelt;
funktioniert gut bei objektorientiertem Denken

Probleme:

- rasch Illusion eines „fertigen“ Produkts
- Fortschrittsmessung schwer möglich (z.B. Meilensteine)
- Gefahr der Lieferung von Prototypen statt Produkten

Schablonenmodell

Vorgehensmethoden – Iterativ-inkrementelles Vorgehen (III)

Schablonenmodell

Erkennen und Anwenden von

- Denkmustern (**patterns**) und
- Problemlöseschablonen (**templates**)

Klassifizieren eines neuen Problems durch
Pattern Matching auf einen Schablonenkatalog



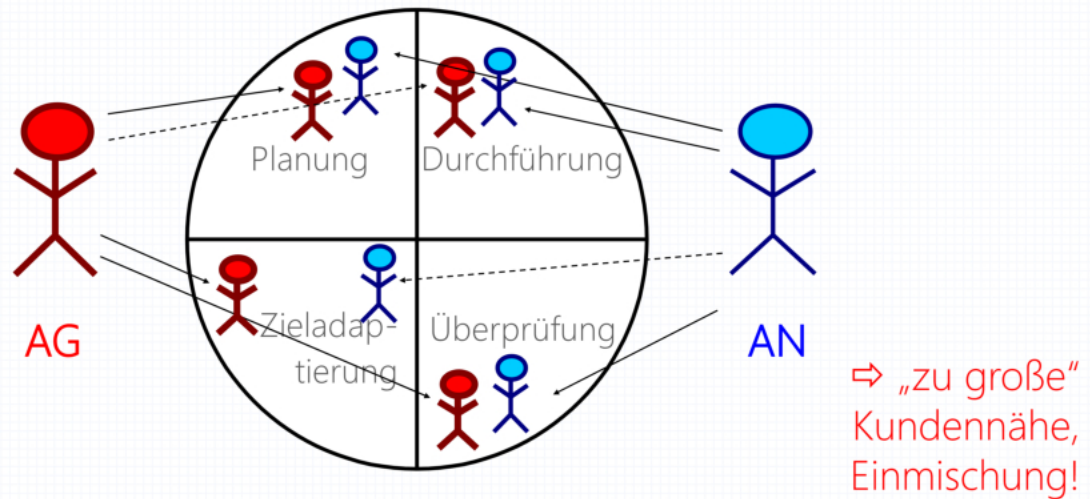
z.B. libraries verwenden!

aber auch bewährte Lösungsstrategien im Kopf haben und anwenden können - man muss das Rad nicht neu erfinden

Iterativ-inkrementelles Vorgehen: 1. Kunden-Integration

Iterativ-inkrementelles Vorgehen: 1. Kunden-Integration

„agil“ (XP-Hype)

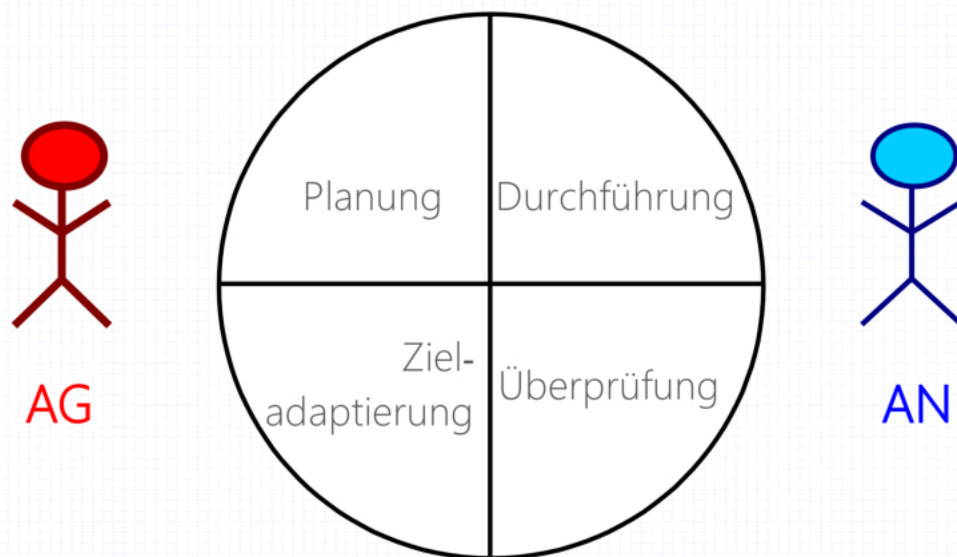


Das war der XP-Hype um 2000, dabei aber zu große Kundennähe und Einmischung!

man braucht eine Pufferfunktion

Iterativ-inkrementelles Vorgehen: 2. Pufferfunktion

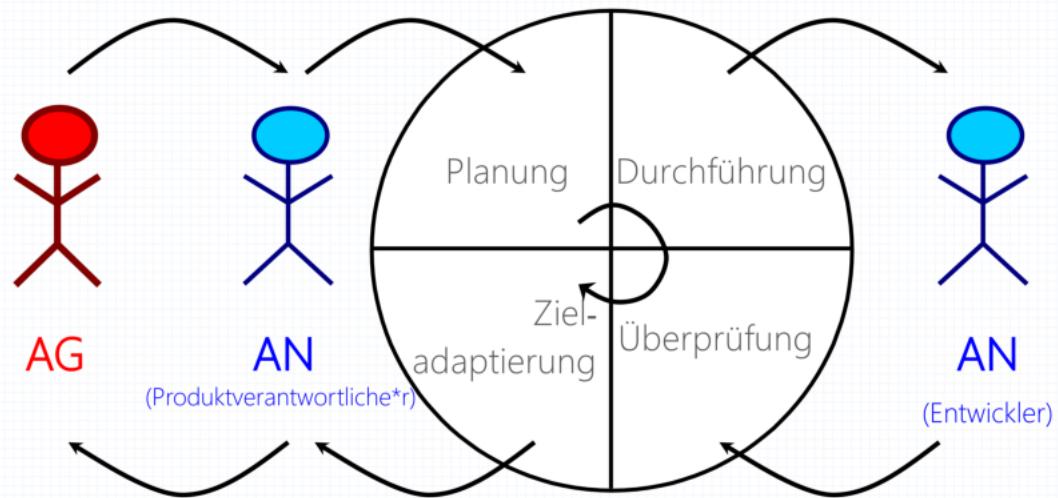
„ideale“ Kundennähe



Dazu nimmt man einen Produktverantwortlichen, der mit dem AG kommuniziert

Iterativ-inkrementelles Vorgehen: 3. Produktverantwortliche*r

„ideale“ Kundennähe



Das ist eine ideale Iteration:

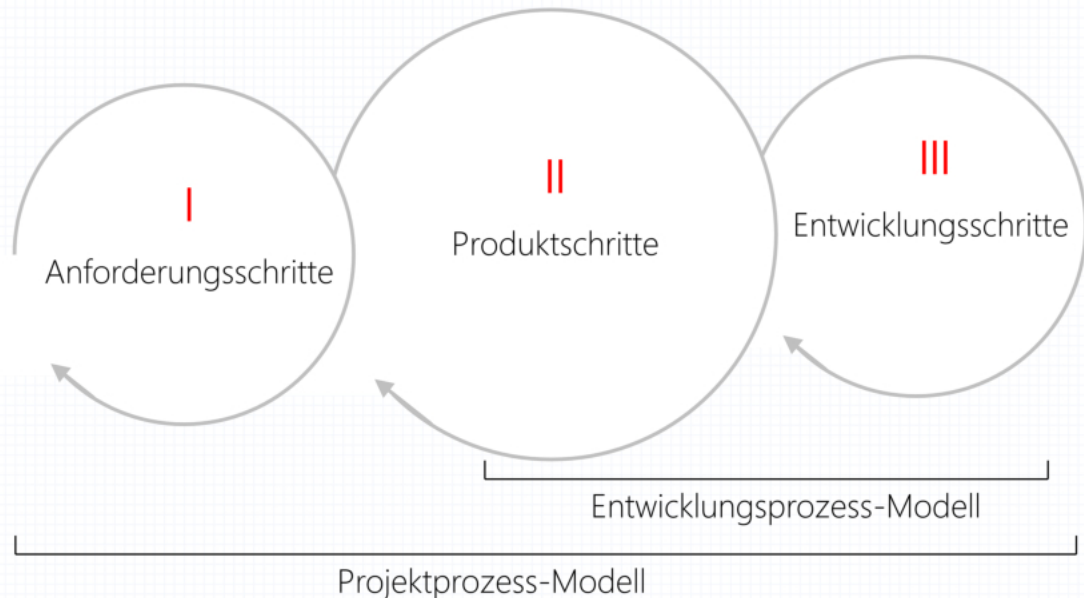
1. Planung (gemeinsam mit Produktverantwortlichen und AG)
2. Durchführung (AN (Entwickler))
3. Überprüfung (AN (Entwickler))
4. Zieladaptierung (gemeinsam mit Produktverantwortlichen und AG)

Dann: neues Inkrement je nach Zieladaptierung

Iterativ-inkrementeller Entwicklungsprozess

Iterativ-inkrementeller Entwicklungsprozess

dreifach rückgekoppelter Prozess:



Moderne („Agile“) Softwareentwicklung

Moderne Softwareentwicklung

Positive Charakteristika:

- + enge Kundeneinbindung
- + iterativ-inkrementelle Entwicklung
- + effiziente Werkzeugnutzung

Negative Charakteristika:

- weniger erfahrene Entwickler (heterogene Teams)
- Aversion gegenüber Management-Information
- kaum Prozessverständnis, geschweige denn -verbesserung



- Aversion: Abneigung (Programmierer wollen programmieren und nicht verwalten)

Agile Softwareentwicklung: Anlass

Agile Softwareentwicklung: Anlass

Software-Prozessmodelle wurden in den Neunziger Jahren immer umfangreicher!

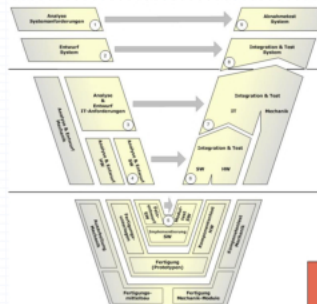


(Grafik: <http://oeag.test.oeag.at>)

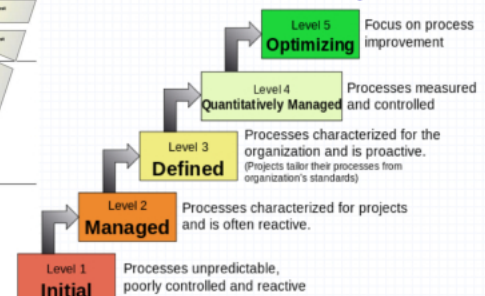
⇒ „Dokumentationsmodelle“

z.B.

- in Deutschland „V-Modell“
- in USA „SW-CMM“



(Grafik: TU München)



(Grafik: Wikipedia)

Agile Softwareentwicklung: Auslöser

Agiles Manifest (nach Beck *et al.*, 2001):

Manifesto for Agile Software Development

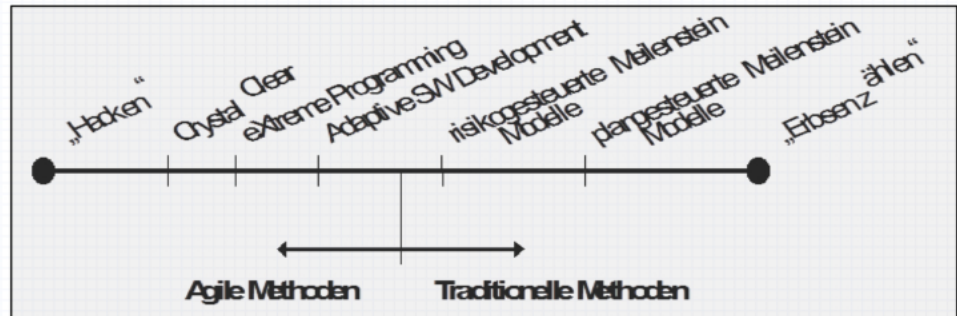
We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over *processes and tools*,
Working software over *comprehensive documentation*,
Customer collaboration over *contract negotiation*,
Responding to change over *following a plan*.

That is, while there is value in the items on the right, we value the items on the left more.

Agile Vorgehensmethoden

Bandbreite des Methodenspektrums:



Paradigmenwechsel durch agile Methoden:

- einheitlicher Entwicklungsprozess nicht definierbar
- Projekte nicht langfristig detailliert planbar
- Projektkontrolle ist ohne Entwicklerkooperation unmöglich

Vorteile agiler Vorgehensmethoden

Vorteile agiler Vorgehensmethoden

- + iterativ-inkrementelle Entwicklung
- + kurze Releasezyklen
- + überprüfbare Qualität
- + intensive Kundeneinbindung
- + intensive Kommunikation im Team
- + Anwendung von Programmierstandards
- + einfaches Design
- + laufende (Code-)Reviews
- + testgesteuerte Entwicklung
- + kontinuierliche Integration

Nachteile agiler Vorgehensmethoden

Nachteile agiler Vorgehensmethoden

- nur hoch qualifizierte Mitarbeiter brauchbar
- schlechte Skalierbarkeit
- mangelnde Transparenz für das Management
- keine Migrationsmöglichkeit aus bestehenden Prozessen
- fehlender Qualitätsplan
- kontinuierliche Anforderungsänderungen
- keine Design-Reviews
- stark inkrementelles Design, fehlende Dokumentation dazu
- Code-Zentriertheit statt Design-Zentriertheit
- fehlende Quellen für Systemtests

Größtes Problem: **fehlendes „Big-Picture“-Design** (Architektur!)

- natürlich nicht so starker Plan aber das ist genau Agilität

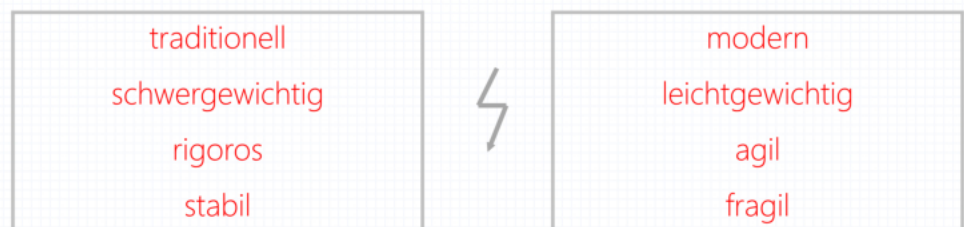
Problem: fehlendes "Big-Picture"-Design (flexible Architektur nötig)

Folgen des Agilen Manifests (I)



(Bild: www.marconetz.de)

Prozesskrieg



„If you want to start a religious or software war,
issue an edict or a manifesto“ (K. Orr, Cutter 2003)

Dokumente & Werkzeuge in agilen Prozessmodellen

Erkenntnisse:

- Auch agile Methoden erzeugen eine **Anzahl** von Dokumenten.
- Dokumente müssen **kompakt** und mit **wenig Aufwand** zu erstellen sein.
- Agile Methoden benötigen häufig **mehr Werkzeugunterstützung** als traditionelle Methoden.
- Werkzeuge müssen **einfach erlernbar** und **rasch handhabbar** sein.

Ein gutes Werkzeug soll den Prozess unterstützen,
nicht vorgeben!

Prozessoptimierung

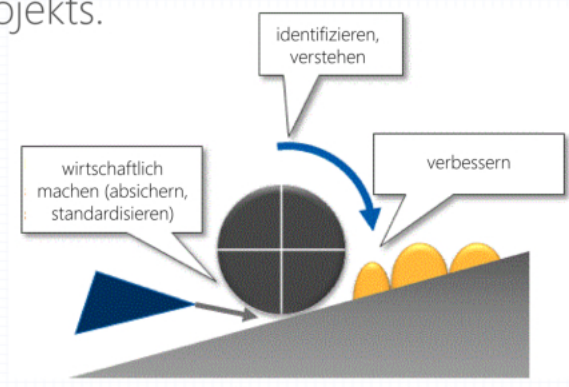
Prozessoptimierung

Motivation: Definierte (Teil-)Prozesse müssen **laufend angepasst und verbessert** werden, damit sich der **Aufwand amortisiert!**

Vor allem bei agilen Methoden Änderungen nicht nur von Projekt zu Projekt, sondern auch während eines Projekts.

Vier wichtige Schritte:

1. Prozess identifizieren,
2. Prozess verstehen,
3. Prozess wirtschaftlich machen,
4. Prozess verbessern.



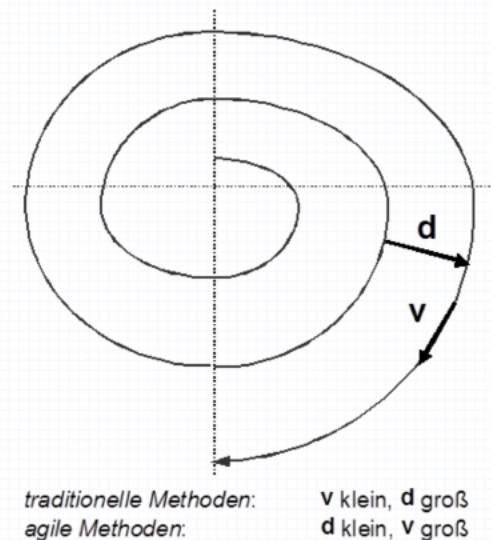
Prozessidentifikation

Prozessidentifikation

Bedeutung des Messens

Zwei Aspekte:

- Messen des **Projektfortschritts**
- Messen des Grades der **Zielerreichung**



Gerade bei agilem Vorgehen Anvisieren eines beweglichen Ziels – Fortbewegung in raschen, kurzen Schritten nötig!

bewegliches Ziel: kleine Anforderungen können sich immer ändern, man muss immer auf Kundenwünsche eingehen

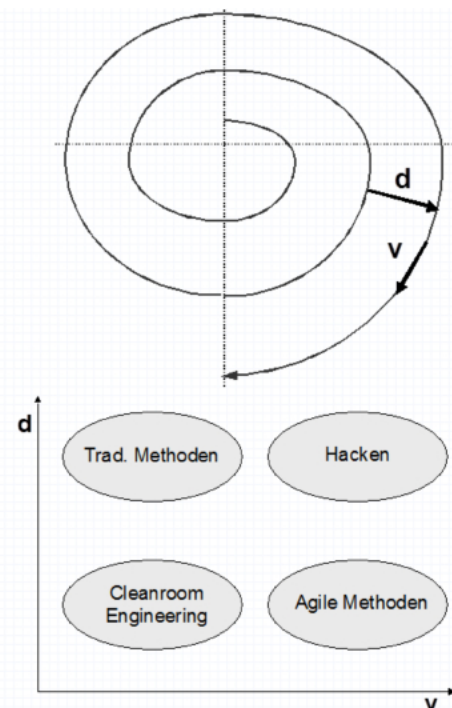
Prozessverständnis

Prozessverständnis

Regelmäßiges Feedback durch **iterativ-inkrementelles Vorgehen**:

- kurze Iterationszyklen
- häufiges **Feedback** vom Auftraggeber
- genauere Ermittlung des **Projektstatus**
- rasche **Adaptierbarkeit**

Durch laufende Validierung auch klare Abgrenzung vom Hackertum!



Prozessökonomie: Traditionelles Vorgehen

Prozessökonomie: Traditionelles Vorgehen

Traditionelle Vorgehensmethoden:

- Vorhersehbarkeit
- Wiederholbarkeit
- Optimierbarkeit
- Streben nach vollständigen Anforderungen
- Plan- und Modell-fokussiert

Risiken:

- Anforderungen ändern sich.
- Aktualisierung der Pläne und Modelle ist sehr aufwändig.

Prozessökonomie: Agiles Vorgehen

Prozessökonomie: Agiles Vorgehen

Agile Vorgehensmethoden:

(AgileAlliance.org, adaptiert durch J. Coldewey)



Agiles Manifest

„Kollaboration statt Formalismus,
kurze Inkremente statt jahrelanger Meditation,
Flexibilität statt Planungsorgien,
Einbindung des Kunden statt Absicherung.“

- Reduktion von Plänen und Modellen auf das Mindestmaß
- (ursprünglich) keine eigene explizite Designphase
- (ursprünglich) keine Prozessverbesserung

Die Balance: Prozessverbesserung

Prozessverbesserung

Gemeinsamkeiten aktueller Vorgehensmethoden:

- Projektziel ist **Vision**, die im Auge behalten werden muss.
- **Kunde** ist laufend eingebunden.
- Realisierung wird nach Festlegung minimaler **Mindestanforderungen** begonnen.
- **Testfälle** werden bereits mit der Anforderung erstellt (vor der Implementierung!).
- **Entwicklung** erfolgt **iterativ-inkrementell** mit kurzen Zyklen.
- Laufende **Risikobehandlung** ist wichtig.
- Nur für Auftraggeber oder Auftragnehmer **essenzielle Dokumente** werden erstellt.
- **Anforderungsänderungen** sind alltäglich und eingeplant.
- **Pläne** werden laufend geprüft und **adaptiert**.
- **Design** ist **einfach**, aber leicht erweiterbar.
- **Tests** werden möglichst **automatisiert** durchgeführt.



(Bild: hundeschule-pfotenteam.de)